

Facultad Ciencias de la Vida y Tecnología

Carrera Ingeniería de Software

6to. Nivel – A

Tema

Inyección SQL

Lenguaje: Python

Motor de base de datos: PostgreSQL

Base de datos: taller_sql

Editor: Visual Studio Code

Terminal: PowerShell / CMD

Vinces Reyes Josue Alexander

Autor

Ing. Daniel Veloz

Tutor

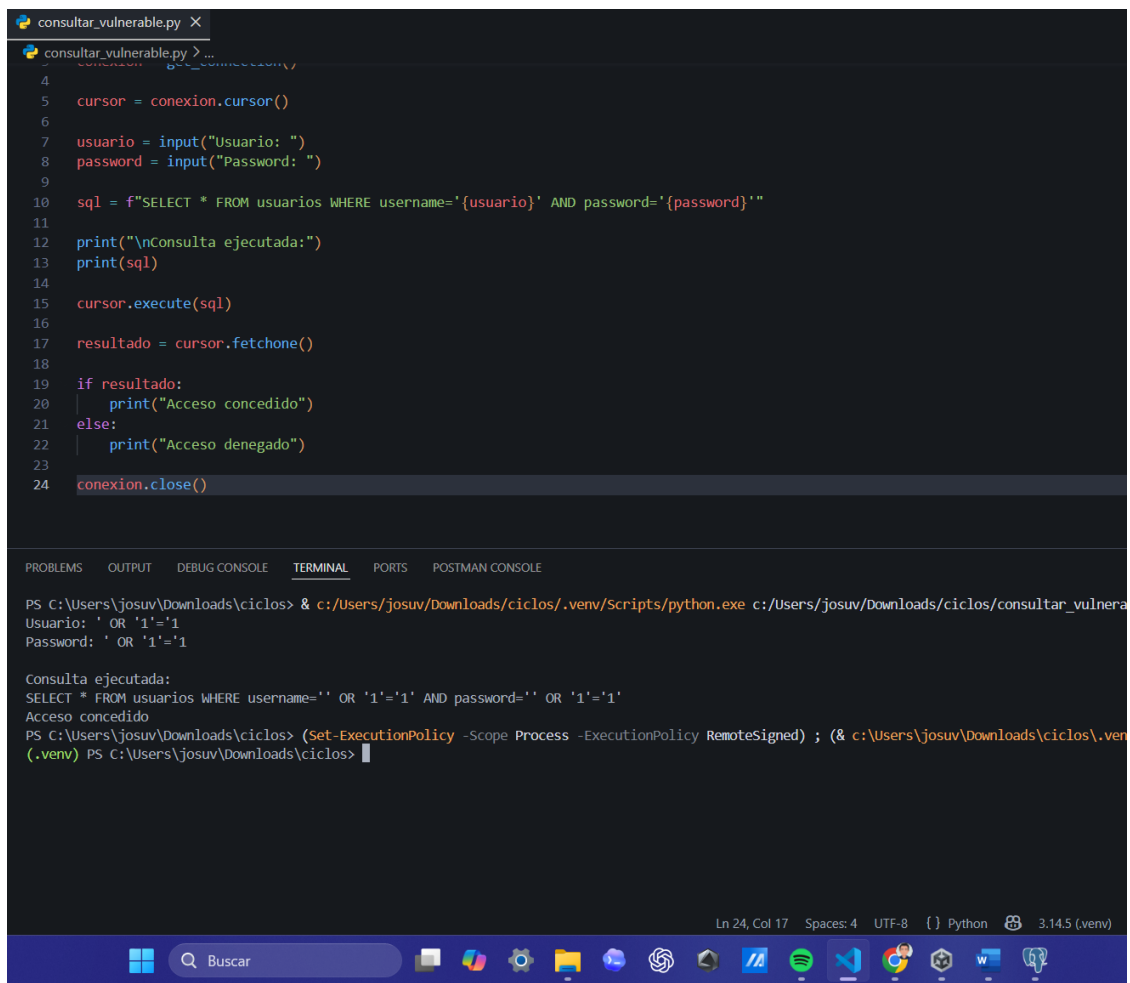
Junio – 2026

Practica vulnerable

La inyección SQL ocurre cuando un atacante inserta código SQL malicioso en campos de entrada (usuario, contraseña, etc.). Si la aplicación no valida o sanitiza estas entradas, el código malicioso se ejecuta en la base de datos.

Ataque 2: Condición siempre verdadera

- Usuario: ' OR '1'='1
- Contraseña: ' OR '1'='1
- Consulta: SELECT * FROM usuarios WHERE username="" OR '1'='1' AND password="" OR '1'='1'
- Acceso sin credenciales válidas



```
consultar_vulnerable.py X
consultar_vulnerable.py > ...
4
5 cursor = conexion.cursor()
6
7 usuario = input("Usuario: ")
8 password = input("Password: ")
9
10 sql = f"SELECT * FROM usuarios WHERE username='{usuario}' AND password='{password}'"
11
12 print("\nConsulta ejecutada:")
13 print(sql)
14
15 cursor.execute(sql)
16
17 resultado = cursor.fetchone()
18
19 if resultado:
20 |     print("Acceso concedido")
21 else:
22 |     print("Acceso denegado")
23
24 conexion.close()
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS POSTMAN CONSOLE

```
PS C:\Users\josuv\Downloads\ciclos> & c:/Users/josuv/Downloads/ciclos/.venv/Scripts/python.exe c:/Users/josuv/Downloads/ciclos/consultar_vulnerable.py
Usuario: ' OR '1'='1
Password: ' OR '1'='1

Consulta ejecutada:
SELECT * FROM usuarios WHERE username='' OR '1'='1' AND password='' OR '1'='1'
Acceso concedido
PS C:\Users\josuv\Downloads\ciclos> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\josuv\Downloads\ciclos\.venv\Scripts\python.exe c:/Users/josuv/Downloads/ciclos/consultar_vulnerable.py)
(.venv) PS C:\Users\josuv\Downloads\ciclos> █
```

Ln 24, Col 17 Spaces: 4 UTF-8 Python 3.14.5 (venv)

Practica segura

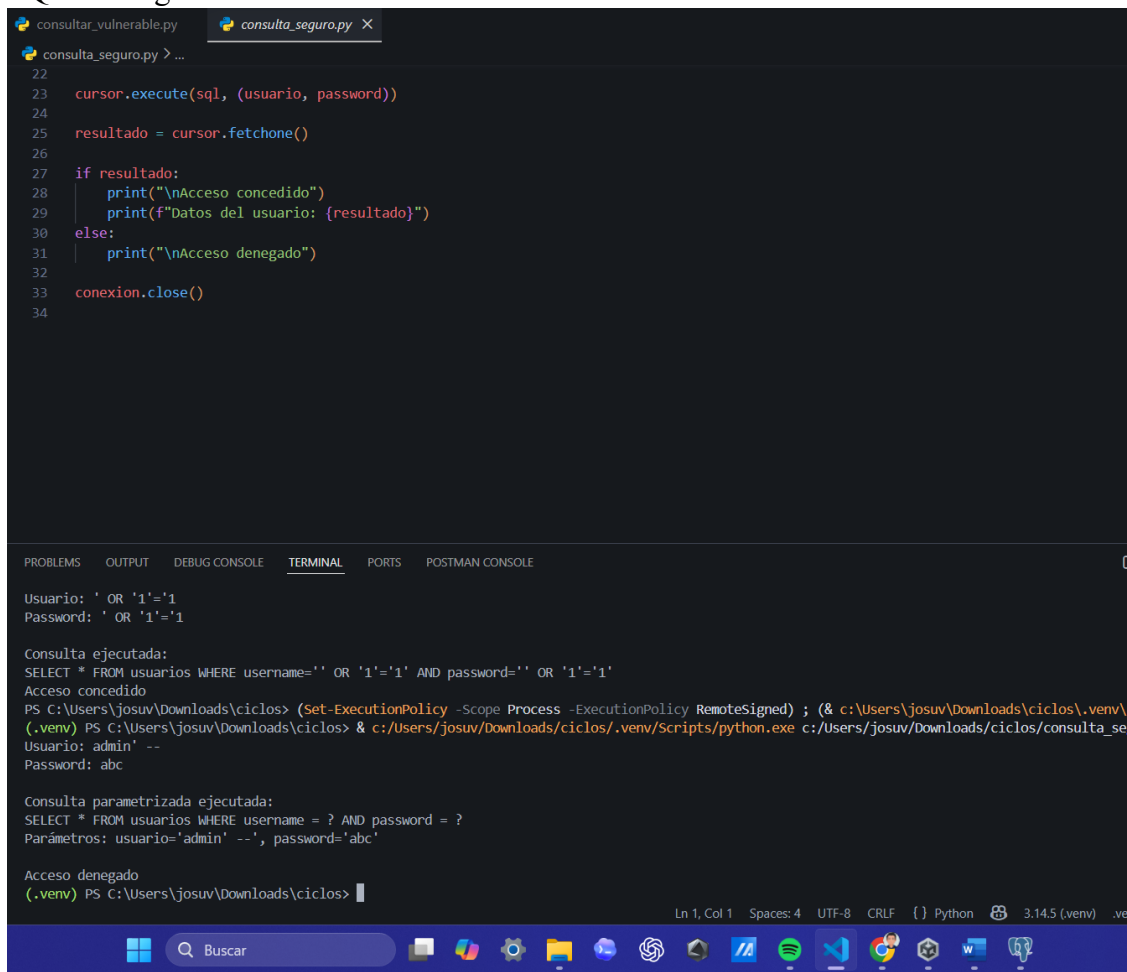
Ahora intenta los mismos ataques de inyección SQL:

Usuario: admin' --

Contraseña: abc

Resultado: Acceso denegado

Porque la consulta parametrizada trata admin' -- como un valor literal, no como código SQL. La seguridad funcionó.



```
consultar_vulnerable.py | consulta_seguro.py X
consulta_seguro.py > ...
22
23     cursor.execute(sql, (usuario, password))
24
25     resultado = cursor.fetchone()
26
27     if resultado:
28         print("\nAcceso concedido")
29         print(f"Datos del usuario: {resultado}")
30     else:
31         print("\nAcceso denegado")
32
33     conexion.close()
34
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

Usuario: ' OR '1'='1
Password: ' OR '1'='1

Consulta ejecutada:
SELECT * FROM usuarios WHERE username='' OR '1'='1' AND password='' OR '1'='1'

Acceso concedido

PS C:\Users\josuv\Downloads\ciclos> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\josuv\Downloads\ciclos\.venv\(.venv) PS C:\Users\josuv\Downloads\ciclos> & c:/Users/josuv/Downloads/ciclos/.venv/Scripts/python.exe c:/Users/josuv/Downloads/ciclos/consulta_se

Usuario: admin' --
Password: abc

Consulta parametrizada ejecutada:
SELECT * FROM usuarios WHERE username = ? AND password = ?
Parámetros: usuario='admin' --', password='abc'

Acceso denegado

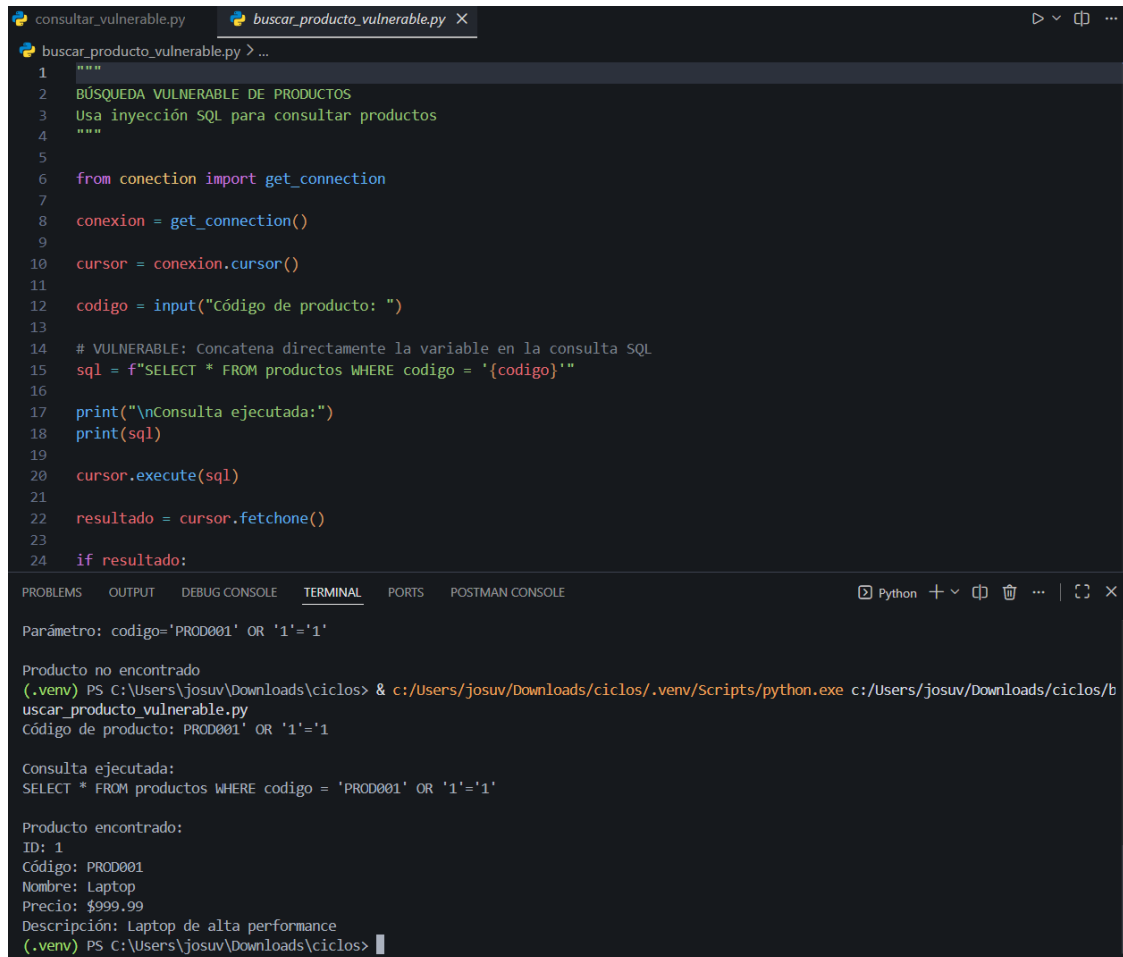
(.venv) PS C:\Users\josuv\Downloads\ciclos> |

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.14.5 (.venv) .ve

Practica 2 vulnerable

Script de búsqueda de productos con ****inyección SQL****.

- Busca por código de producto
- Vulnerable a `PROD001' OR '1'='1` (retorna todos)
- Muestra detalles del producto encontrado
- Función: Demostrar inyección en búsquedas



```
consultar_vulnerable.py  buscar_producto_vulnerable.py X
1  """
2  BÚSQUEDA VULNERABLE DE PRODUCTOS
3  Usa inyección SQL para consultar productos
4  """
5
6  from conection import get_connection
7
8  conexion = get_connection()
9
10 cursor = conexion.cursor()
11
12 codigo = input("Código de producto: ")
13
14 # VULNERABLE: Concatena directamente la variable en la consulta SQL
15 sql = f"SELECT * FROM productos WHERE codigo = '{codigo}'"
16
17 print("\nConsulta ejecutada:")
18 print(sql)
19
20 cursor.execute(sql)
21
22 resultado = cursor.fetchone()
23
24 if resultado:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE Python + - 🔍 🗑️ ... | 🔄 🔍 X

Parámetro: codigo='PROD001' OR '1'='1'

Producto no encontrado

(.venv) PS C:\Users\josuv\Downloads\ciclos> & c:/Users/josuv/Downloads/ciclos/.env/Scripts/python.exe c:/Users/josuv/Downloads/ciclos/b
uscar_producto_vulnerable.py
Código de producto: PROD001' OR '1'='1

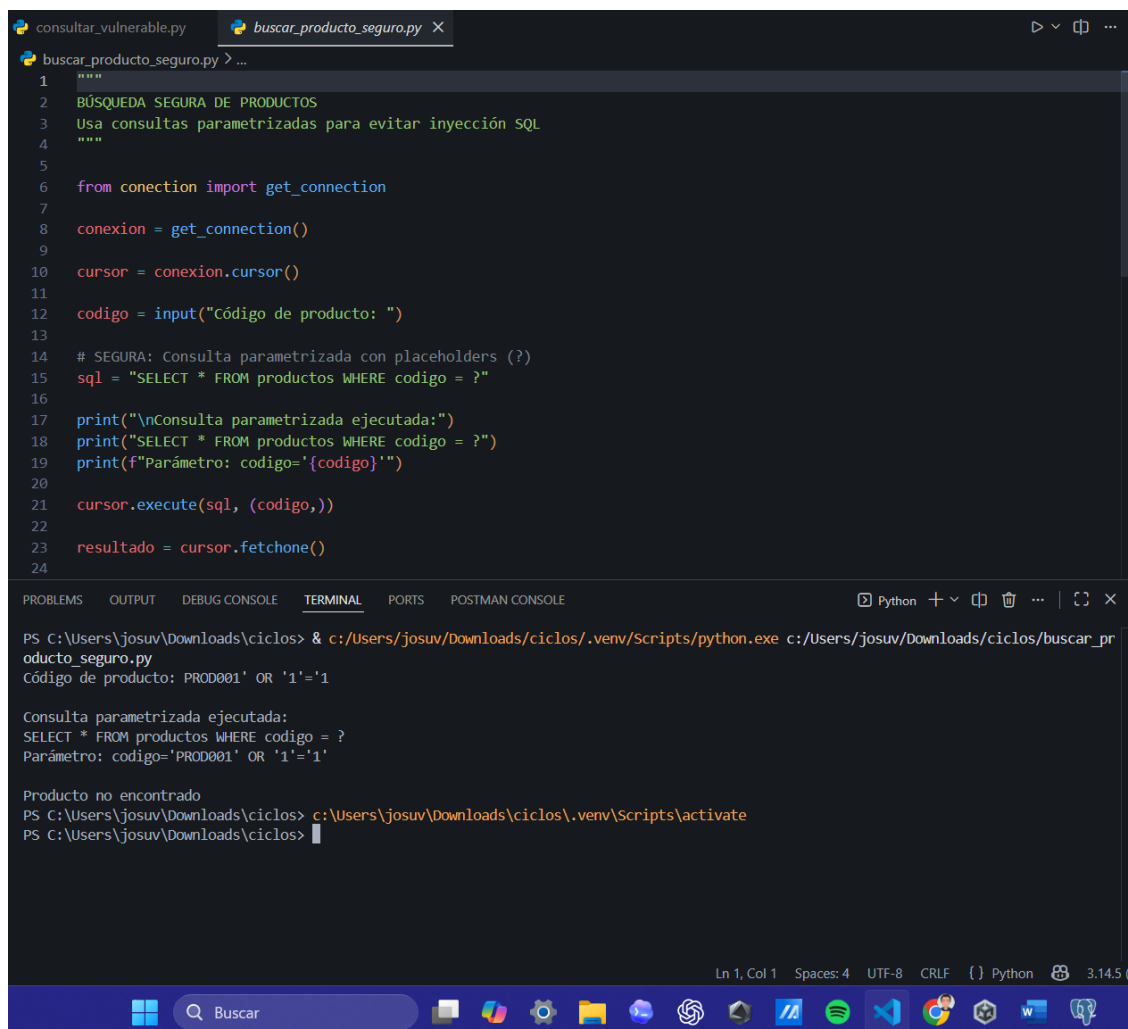
Consulta ejecutada:
SELECT * FROM productos WHERE codigo = 'PROD001' OR '1'='1'

Producto encontrado:
ID: 1
Código: PROD001
Nombre: Laptop
Precio: \$999.99
Descripción: Laptop de alta performance
(.venv) PS C:\Users\josuv\Downloads\ciclos>

Practica 2 seguro

Script de búsqueda de productos ****protegido****.

- Usa consultas parametrizadas
- Bloquea intentos de inyección SQL
- Mismo formato que versión vulnerable pero seguro
- Función: Búsqueda segura de productos



```
consultar_vulnerable.py  buscar_producto_seguro.py X
buscar_producto_seguro.py > ...
1  """
2  BÚSQUEDA SEGURA DE PRODUCTOS
3  Usa consultas parametrizadas para evitar inyección SQL
4  """
5
6  from conection import get_connection
7
8  conexion = get_connection()
9
10 cursor = conexion.cursor()
11
12 codigo = input("Código de producto: ")
13
14 # SEGURA: Consulta parametrizada con placeholders (?)
15 sql = "SELECT * FROM productos WHERE codigo = ?"
16
17 print("\nConsulta parametrizada ejecutada:")
18 print("SELECT * FROM productos WHERE codigo = ?")
19 print(f"Parámetro: codigo='{codigo}'")
20
21 cursor.execute(sql, (codigo,))
22
23 resultado = cursor.fetchone()
24
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE Python + - 🔍 🗑️ ... | 🔄 ✕

```
PS C:\Users\josuv\Downloads\ciclos> & c:/Users/josuv/Downloads/ciclos/.venv/Scripts/python.exe c:/Users/josuv/Downloads/ciclos/buscar_producto_seguro.py
Código de producto: PROD001' OR '1'='1

Consulta parametrizada ejecutada:
SELECT * FROM productos WHERE codigo = ?
Parámetro: codigo='PROD001' OR '1'='1'

Producto no encontrado
PS C:\Users\josuv\Downloads\ciclos> c:/Users/josuv/Downloads/ciclos/.venv/Scripts/activate
PS C:\Users\josuv\Downloads\ciclos>
```

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.14.5